

ASSIGNMENT-11.3

Name: V.Sai Anand

Ht.no: 2303A51310

Batch: 05

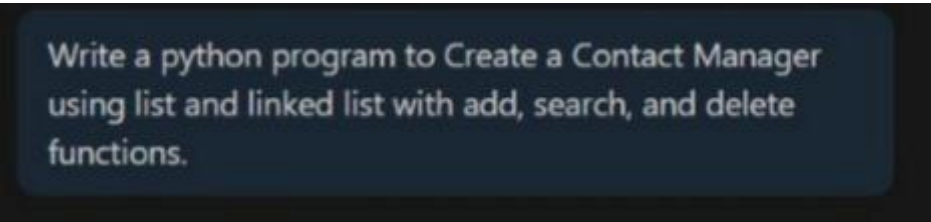
Task 1: Smart Contact Manager (Arrays & Linked Lists)

Scenario

SR University's student club requires a simple Contact Manager Application to store members' names and phone numbers. The system should support efficient addition, searching, and deletion of contacts.

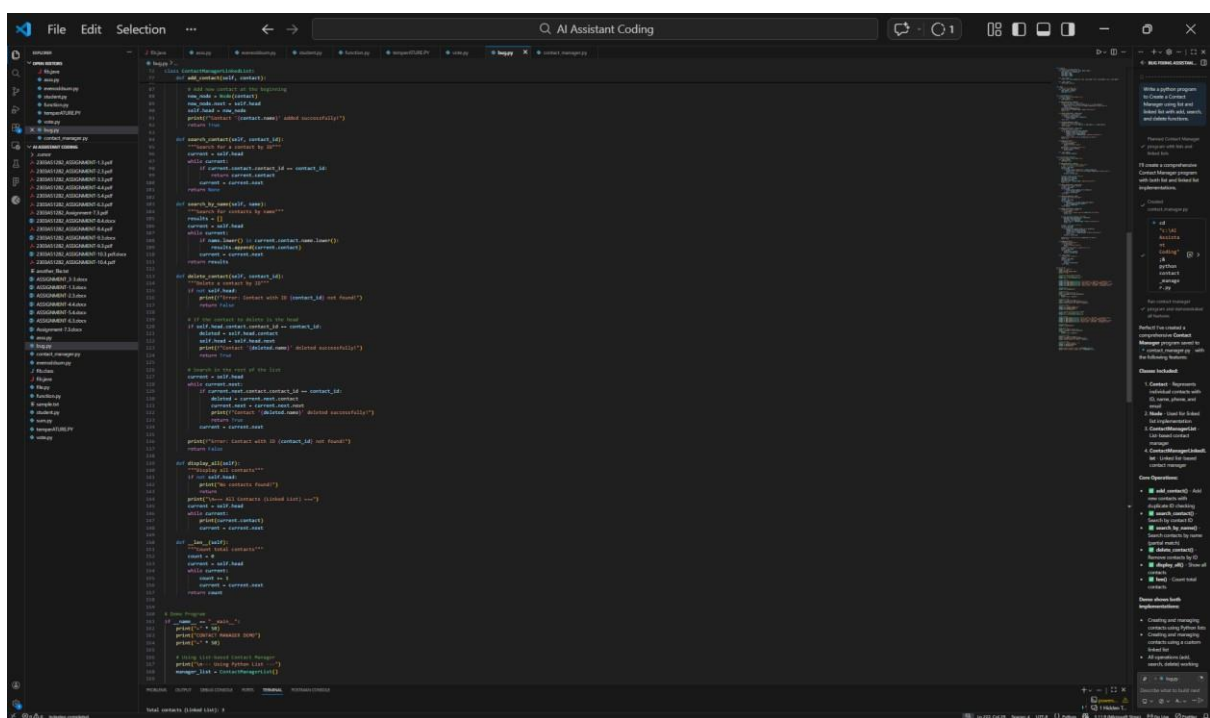
Prompt:

Write a python program to Create a Contact Manager using list and linked list with add, search, and delete functions.



Write a python program to Create a Contact Manager using list and linked list with add, search, and delete functions.

Code:



- In an array, adding at the end is fast, but inserting in the middle is slow because elements must shift.

- In a linked list, insertion is fast because no shifting is needed.
- Searching takes the same time in both (you must check each element).
- Deleting in an array is slower due to shifting elements.
- Linked list is better for frequent insertions and deletions.

Task 2: Library Book Search System (Queues & Priority Queues)

Scenario

The SRU Library manages book borrow requests. Students and faculty submit requests, but faculty requests must be prioritized over student requests.

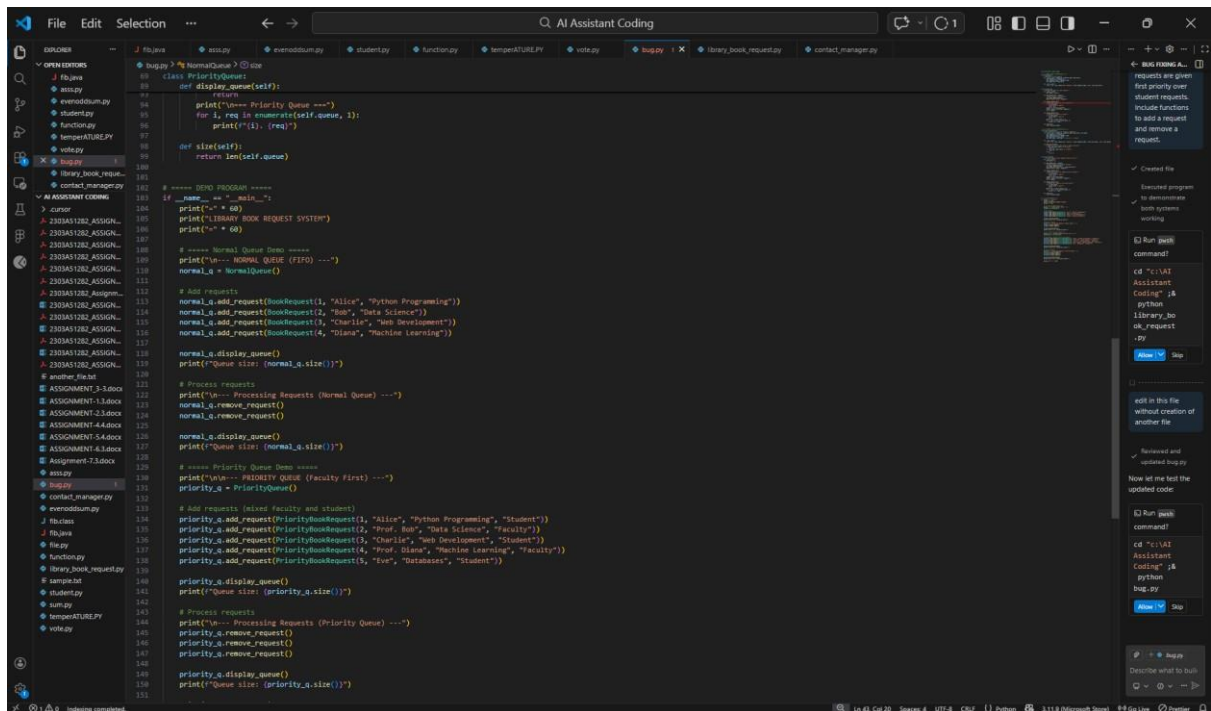
Prompt:

Write a Python program for a library book request system. First, make a normal queue where requests are handled in the order they come. Then, make another version where faculty requests are given first priority over student requests. Include functions to add a request and remove a request.

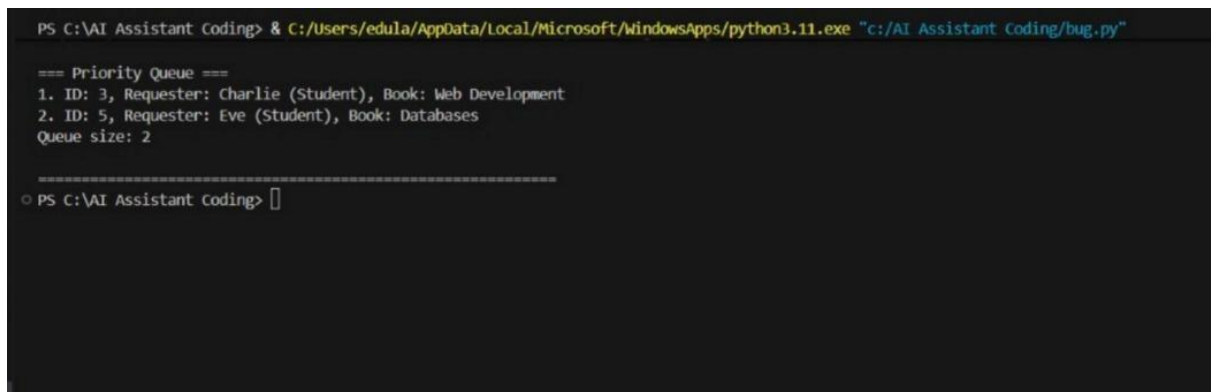
Write a Python program for a library book request system. First, make a normal queue where requests are handled in the order they come. Then, make another version where faculty requests are given first priority over student requests. Include functions to add a request and remove a request.

Code:

The image shows a VS Code editor window with a file named 'book_request.py' open. The code implements a priority queue system for book requests. It starts with a 'PriorityQueue' class that has a 'queue' attribute. The 'add_request' method appends a request to the queue and sorts it by priority. The 'remove_request' method removes the highest priority request. The 'display_queue' method prints all requests in priority order. The 'main' function creates a 'NormalQueue' object and processes a list of requests with different priorities. The requests are: 'Alice', 'Python Programming' (priority 1); 'Bob', 'Data Science' (priority 2); 'Charlie', 'Web Development' (priority 3); and 'Diana', 'Machine Learning' (priority 4). The output shows the requests being processed in order of priority: Charlie's request first, followed by Bob's, then Alice's, and finally Diana's.



Output:



Explanation:

- Queue (FIFO) → First request comes, first served. (If a student requests first, they get the book first.)
- Priority Queue → Faculty requests are served before students, even if they come later.
- enqueue() → Adds a request to the system.

- `dequeue()` → Removes and processes the next request.

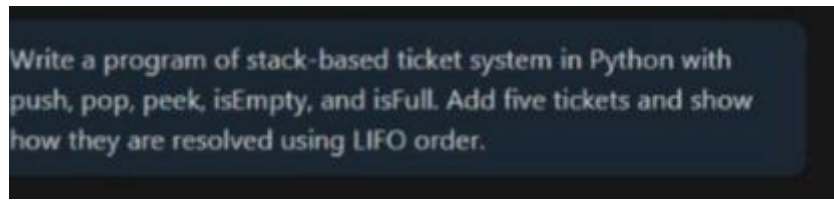
Task 3: Emergency Help Desk (Stack Implementation)

Scenario

SR University's IT Help Desk receives technical support tickets from students and staff. While tickets are received sequentially, issue escalation follows a Last-In, First-Out (LIFO) approach.

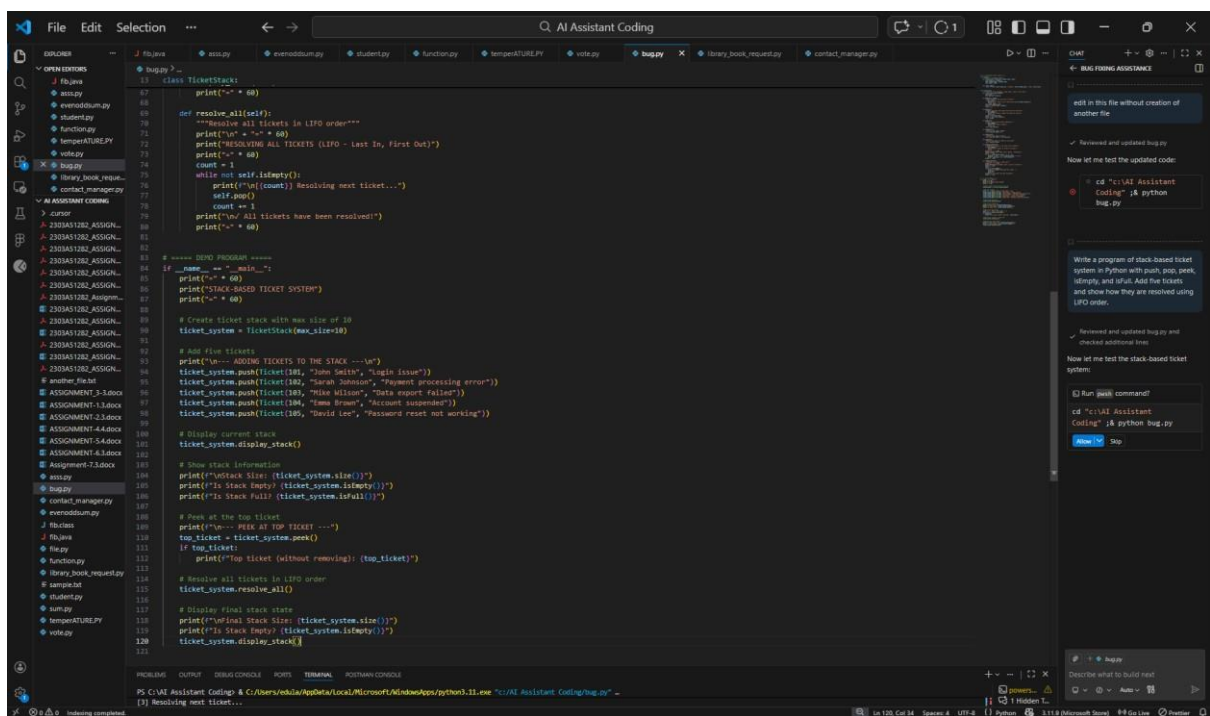
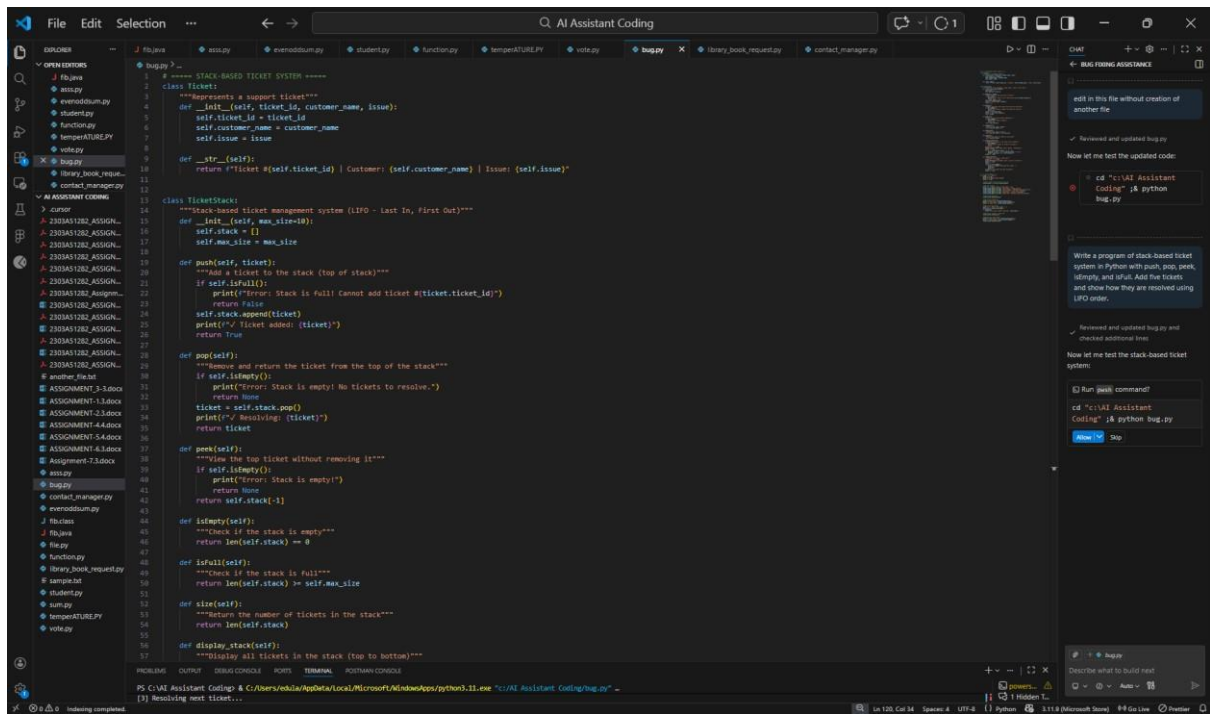
Prompt:

Write a program of stack-based ticket system in Python with `push`, `pop`, `peek`, `isEmpty`, and `isFull`. Add five tickets and show how they are resolved using LIFO order.

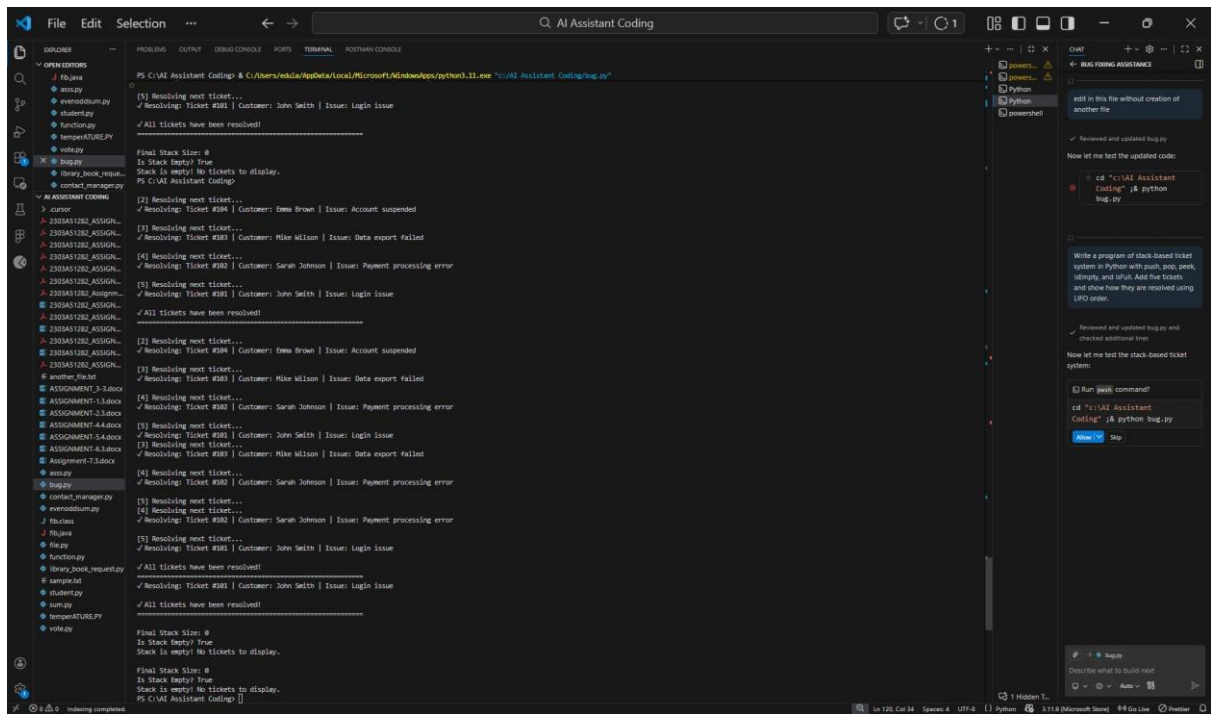


Write a program of stack-based ticket system in Python with `push`, `pop`, `peek`, `isEmpty`, and `isFull`. Add five tickets and show how they are resolved using LIFO order.

Code:



Output:



Explanation:

The program uses a stack to manage help desk tickets.

A stack works in last in, first solved order.

When a new ticket is raised, it is added to the top. When solving a ticket, the most recent one is handled first.

The program can also check if there are no tickets left or if the stack is full.

Task 4: Hash Table

Objective

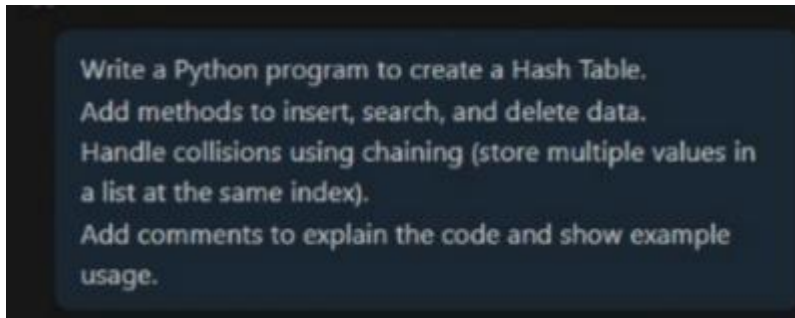
To implement a Hash Table and understand collision handling.

Prompt:

Write a Python program to create a Hash Table.

Add methods to insert, search, and delete data. Handle collisions using chaining (store multiple values in a list at the same index).

Add comments to explain the code and show example usage.



Code:

```
1 # Create Hash Table with CHAINING collision handling using
2 # key-value pair class for storing data
3 class KeyValuePair:
4     """Represents a key-value pair in the hash table"""
5     def __init__(self, key, value):
6         self.key = key
7         self.value = value
8
9     def __str__(self):
10         return f"({self.key}, {self.value})"
11
12 class HashTable:
13     """Hash Table Implementation using chaining to handle collisions.
14     Chaining stores multiple key-value pairs that hash to the same index in a linked list.
15     """
16     def __init__(self, size=10):
17         """Initialize the hash table
18         Args:
19             size (int): Number of buckets in the hash table
20         """
21         self.size = size
22         # Create an array of empty lists (chains) for collision handling
23         self.table = [[] for _ in range(size)]
24         self.table_size = 0
25
26     def hash_function(self, key):
27         """Hash function to determine the index for a given key
28         Uses simple modulo operation: hash(key) % table_size
29         Args:
30             key: The key to hash
31         Returns:
32             int: Index in the hash table
33         """
34         # Convert key to string and sum ASCII values for different key types
35         key_str = str(key)
36         hash_value = sum(ord(char) for char in key_str)
37         return hash_value % self.size
38
39     def insert(self, key, value):
40         """Insert a key-value pair into the hash table
41         If key already exists, update the value
42         If collision occurs, add to the chain (list) at that index
43         Args:
44             key: The key to insert
45             value: The value associated with the key
46         Returns:
47             bool: True if insertion successful, False otherwise
48         """
49         # Get the hash index
50         index = self.hash_function(key)
51         chain = self.table[index]
52
53         # Check if key already exists in the chain and update if found
54         for i, pair in enumerate(chain):
55             if pair.key == key:
56                 chain[i] = (key, value)
57                 return True
58
59         # Add new key-value pair to the chain
60         chain.append((key, value))
61         self.table_size += 1
62         return True
63
64     def search(self, key):
65         """Search for a key in the hash table
66         Args:
67             key: The key to search for
68         Returns:
69             tuple: (key, value) if found, None otherwise
70         """
71         index = self.hash_function(key)
72         chain = self.table[index]
73
74         for pair in chain:
75             if pair.key == key:
76                 return pair
77
78         return None
79
80     def delete(self, key):
81         """Delete a key-value pair from the hash table
82         Args:
83             key: The key to delete
84         Returns:
85             bool: True if deletion successful, False otherwise
86         """
87         index = self.hash_function(key)
88         chain = self.table[index]
89
90         for i, pair in enumerate(chain):
91             if pair.key == key:
92                 chain.pop(i)
93                 self.table_size -= 1
94                 return True
95
96         return False
97
98 class ContactManager:
99     """A simple contact manager using the HashTable"""
100     def __init__(self):
101         self.contacts = HashTable()
102
103     def add_contact(self, name, phone):
104         """Add a new contact"""
105         self.contacts.insert(name, phone)
106
107     def get_contact(self, name):
108         """Get contact information by name"""
109         return self.contacts.search(name)
110
111     def delete_contact(self, name):
112         """Delete a contact"""
113         self.contacts.delete(name)
114
115 # Example usage
116 if __name__ == "__main__":
117     manager = ContactManager()
118     manager.add_contact("John Doe", "123-456-7890")
119     manager.add_contact("Jane Smith", "987-654-3210")
120     manager.add_contact("John Doe", "098-765-4321") # Collision example
121
122     print("Contacts in the Hash Table:")
123     for name, phone in manager.get_contacts():
124         print(f"{name}: {phone}")
125
126     # Search for a contact
127     contact = manager.get_contact("John Doe")
128     if contact:
129         print(f"Found contact: {contact}")
130     else:
131         print("Contact not found")
132
133     # Delete a contact
134     manager.delete_contact("Jane Smith")
135     print("Deleted contact: Jane Smith")
136
137     # Final state
138     print("Final Contacts in the Hash Table:")
139     for name, phone in manager.get_contacts():
140         print(f"{name}: {phone}")
```

```
def insert(self, key, value):
    """Insert a key-value pair into the hash table and update if found"""
    for i, pair in enumerate(self._table):
        if pair is None:
            self._table[i] = (key, value)
            return True
        elif pair[0] == key:
            self._table[i] = (key, value)
            return True
    # If key not found, add new key-value pair to the chain
    chain = self._table[self._hash(key)]
    chain.append((key, value))
    print(f"Inserted: {key} = {value} at index {self._hash(key)}")
    return True

def search(self, key):
    """Search for a key in the hash table"""
    # Search for the key in the hash table
    args = {
        "key": key,
        "value": None
    }
    # Returns:
    # The value associated with the key, or None if not found
    # Get the hash index
    index = self._hash(key)
    chain = self._table[index]
    # Search for the key in the chain
    for pair in chain:
        if pair[0] == key:
            return pair[1]
    # Key not found
    print(f"Key '{key}' not found in the hash table")
    return None

def delete(self, key):
    """Delete a key-value pair from the hash table"""
    # Delete a key-value pair from the hash table
    args = {
        "key": key,
        "value": None
    }
    # Returns:
    # bool: True if deletion successful, False if key not found
    # Get the hash index
    index = self._hash(key)
    chain = self._table[index]
    # Search for and remove the key from the chain
    for i, pair in enumerate(chain):
        if pair[0] == key:
            del chain[i]
            self._table[index] = chain
            print(f"Deleted: {key} = {pair[1]} (from index {index})")
            return True
    # Key not found
    print(f"Key '{key}' not found in the hash table")
    return False

def display(self):
    """Display the hash table"""
    print("Hash Table Contents:")
    for i, pair in enumerate(self._table):
        if pair is None:
            print(f"Index {i}: None")
        else:
            print(f"Index {i}: {pair[0]} = {pair[1]}")
    print("End of Hash Table Contents")
```

```
def get_all_items(self):
    """Get all key-value pairs in the hash table"""
    all_items = []
    for i, pair in enumerate(self._table):
        if pair is None:
            continue
        all_items.append(pair)
    return all_items

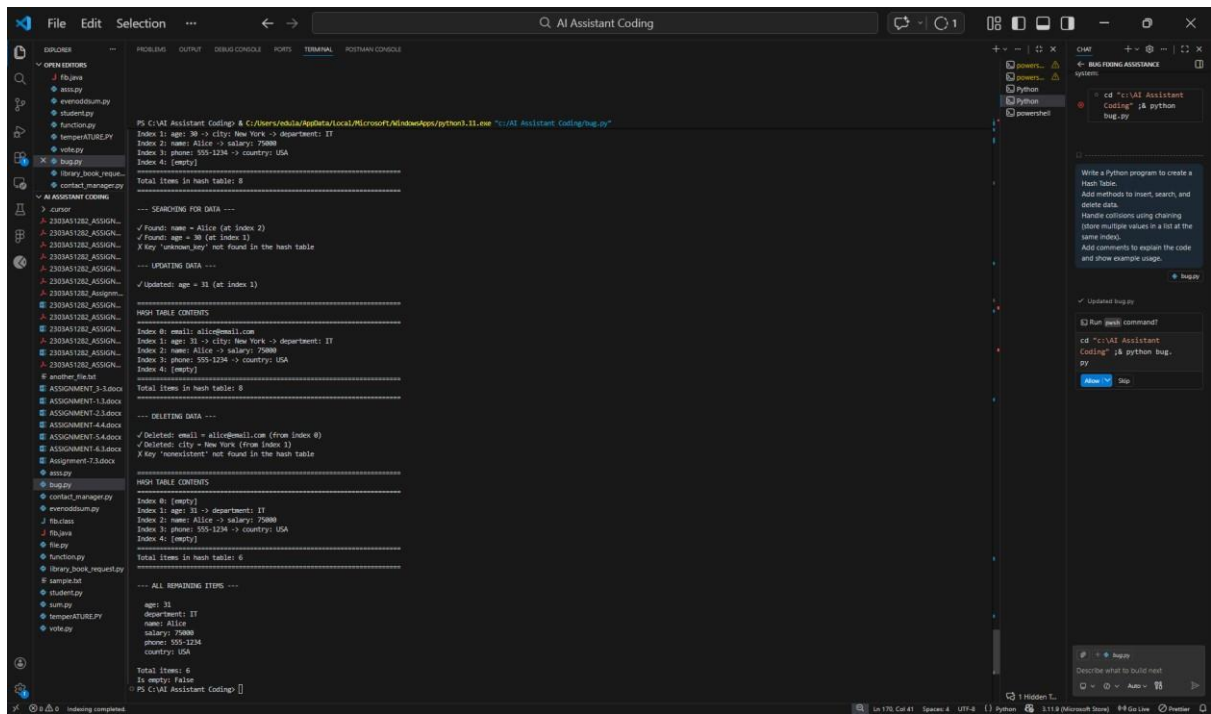
def insert(self, key, value):
    """Insert a key-value pair into the hash table and update if found"""
    # Insert a key-value pair into the hash table
    args = {
        "key": key,
        "value": value
    }
    # Returns:
    # bool: True if insertion successful, False if key already exists
    # Get the hash index
    index = self._hash(key)
    chain = self._table[index]
    # Search for the key in the chain
    for pair in chain:
        if pair[0] == key:
            # Key already exists, update value
            pair[1] = value
            return True
    # Key not found, add new key-value pair to the chain
    chain.append((key, value))
    print(f"Inserted: {key} = {value} at index {self._hash(key)}")
    return True

def search(self, key):
    """Search for a key in the hash table"""
    # Search for the key in the hash table
    args = {
        "key": key,
        "value": None
    }
    # Returns:
    # The value associated with the key, or None if not found
    # Get the hash index
    index = self._hash(key)
    chain = self._table[index]
    # Search for the key in the chain
    for pair in chain:
        if pair[0] == key:
            return pair[1]
    # Key not found
    print(f"Key '{key}' not found in the hash table")
    return None

def delete(self, key):
    """Delete a key-value pair from the hash table"""
    # Delete a key-value pair from the hash table
    args = {
        "key": key,
        "value": None
    }
    # Returns:
    # bool: True if deletion successful, False if key not found
    # Get the hash index
    index = self._hash(key)
    chain = self._table[index]
    # Search for and remove the key from the chain
    for i, pair in enumerate(chain):
        if pair[0] == key:
            del chain[i]
            self._table[index] = chain
            print(f"Deleted: {key} = {pair[1]} (from index {index})")
            return True
    # Key not found
    print(f"Key '{key}' not found in the hash table")
    return False

def display(self):
    """Display the hash table"""
    print("Hash Table Contents:")
    for i, pair in enumerate(self._table):
        if pair is None:
            print(f"Index {i}: None")
        else:
            print(f"Index {i}: {pair[0]} = {pair[1]}")
    print("End of Hash Table Contents")
```

Output:



Explanation:

- A Hash Table stores data using a key and value.
- A hash function decides where to store the data.
- Sometimes two keys go to the same place. This is called a collision.
- To solve collisions, we use chaining, meaning we store multiple items in a list at the same index.
- The program should allow adding, finding, and removing data correctly.

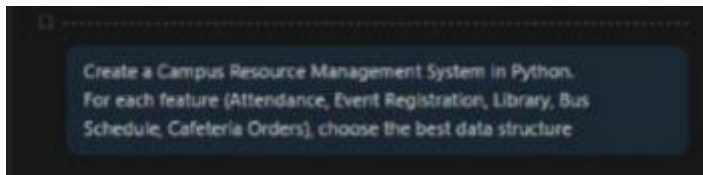
Task 5: Real-Time Application Challenge Scenario

Design a Campus Resource Management System with the following features:

- Student Attendance Tracking
- Event Registration System

- Library Book Borrowing
- Bus Scheduling System
- Cafeteria Order Queue **Prompt:**

Create a Campus Resource Management System in Python. For each feature (Attendance, Event Registration, Library, Bus Schedule, Cafeteria Orders), choose the best data structure

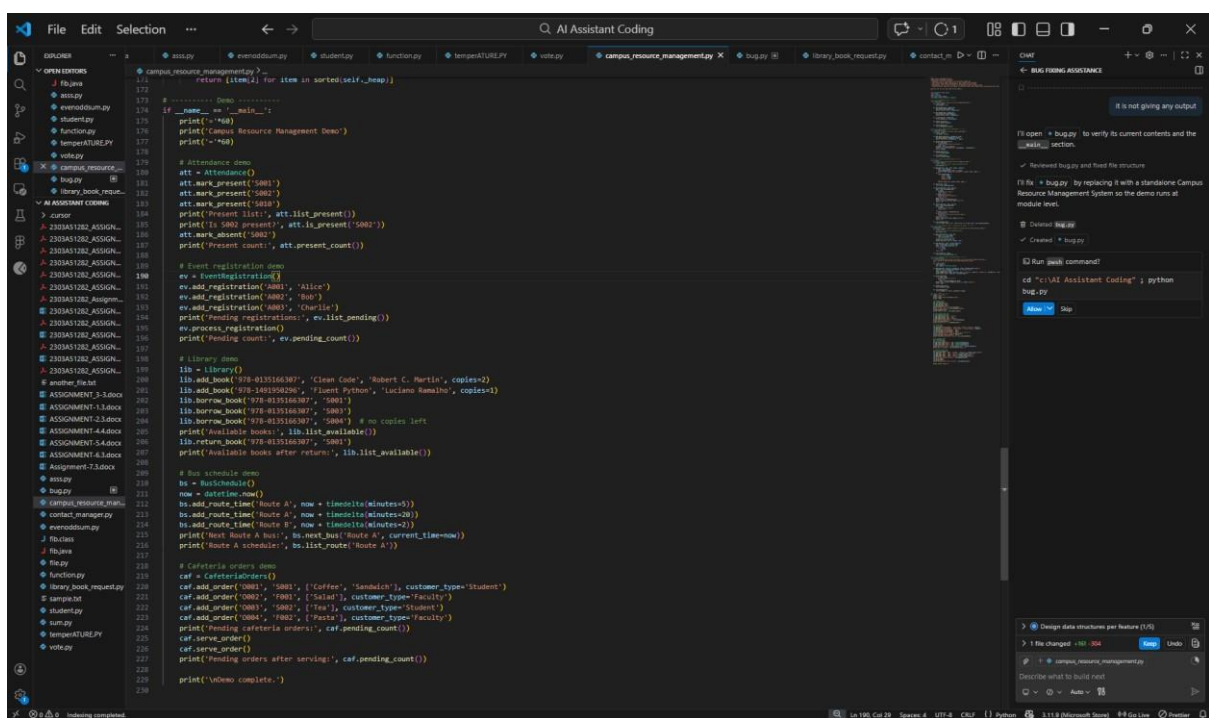


Code:

```

1  # Campus Resource Management System
2  # Features and chosen data structures:
3  # - Attendance: set (O(1) add/remove) to track present student IDs
4  # - Event Registration: deque (FIFO queue) for fair registration processing
5  # - Library: dict (hash table) mapping ISBN to book record for fast lookup
6  # - Bus Schedule: dict of routes -> sorted list of departure times (list kept sorted)
7  # - Cafeteria Orders: heapq (priority queue) to prioritize faculty over students while preserving arrival order
8
9  Run this file to see a small demo of each feature.
10
11  """
12
13  from collections import deque
14  import heapq
15  import heapq
16  from bisect import bisect
17  from datetime import datetime, timedelta
18
19  # ----- Attendance (set) -----
20  class Attendance:
21      """Track attendance using a set for O(1) add/remove/check."""
22      def __init__(self):
23          self.present = set()
24
25      def mark_present(self, student_id):
26          self.present.add(student_id)
27          print(f"Marked present: {student_id}")
28
29      def mark_absent(self, student_id):
30          self.present.discard(student_id)
31          print(f"Marked absent: {student_id}")
32
33      def is_present(self, student_id):
34          return student_id in self.present
35
36      def present_count(self):
37          return len(self.present)
38
39      def list_present(self):
40          return sorted(self.present)
41
42  # ----- Event Registration (FIFO queue) -----
43  class EventRegistration:
44      """Register attendees in arrival order using deque."""
45      def __init__(self):
46          self.queue = deque()
47
48      def add_registration(self, attendee_id, name):
49          self.queue.append((attendee_id, name))
50          print(f"Registered: {attendee_id} - {name}")
51
52      def process_registration(self):
53          if not self.queue:
54              print("No registrations to process.")
55              return None
56          attendee = self.queue.popleft()
57          print(f"Processed registration: {attendee[0]} - {attendee[1]}")
58          return attendee
59
60      def pending_count(self):
61          return len(self.queue)

```



```
PROBLEMS OUTPUT DEBUG CONSOLE PORTS TERMINAL POSTMAN CONSOLE

PS C:\AI Assistant Coding> & C:/Users/edula/AppData/Local/Microsoft/WindowsApps/python3.11.exe "c:/AI Assistant Coding/bug.py"
Is empty: False
PS C:\AI Assistant Coding> & C:/Users/edula/AppData/Local/Microsoft/WindowsApps/python3.11.exe "c:/AI Assistant Coding/bug.py"
PS C:\AI Assistant Coding> & C:/Users/edula/AppData/Local/Microsoft/WindowsApps/python3.11.exe "c:/AI Assistant Coding/campus_resource_management.py"
=====
Campus Resource Management Demo
=====
Marked present: 5001
Marked present: 5002
Marked present: 5010
Present list: ['5001', '5002', '5010']
Is 5002 present? True
Marked absent: 5002
Present count: 2
Registered: A001 - Alice
Registered: A002 - Bob
Registered: A003 - Charlie
Pending registrations: [('A001', 'Alice'), ('A002', 'Bob'), ('A003', 'Charlie')]
Processed registration: A001 - Alice
Pending count: 2
Added book: Clean Code (ISBN: 978-0135166307).
Added book: Fluent Python (ISBN: 978-1491958296).
5001 borrowed Clean Code
5003 borrowed Clean Code
No copies available.
Available books: [('978-0135166307', 'Clean Code', 0), ('978-1491958296', 'Fluent Python', 1)]
5001 returned Clean Code
Available books after return: [('978-0135166307', 'Clean Code', 1), ('978-1491958296', 'Fluent Python', 1)]
Added bus time for Route A: 2026-02-18 10:42:24.367227
Added bus time for Route A: 2026-02-18 10:57:24.367227
Added bus time for Route B: 2026-02-18 10:39:24.367227
Next Route A bus: 2026-02-18 10:42:24.367227
Route A schedule: [datetime.datetime(2026, 2, 18, 10, 42, 24, 367227), datetime.datetime(2026, 2, 18, 10, 57, 24, 367227)]
Order added: 0001 (Student)
Order added: 0002 (Faculty)
Order added: 0003 (Student)
Order added: 0004 (Faculty)
Pending cafeteria orders: 4
Serving order: 0002 (Faculty)
Serving order: 0004 (Faculty)
Pending orders after serving: 2

Demo complete.
PS C:\AI Assistant Coding>
```

Explanation:

Library Book Borrowing using a queue:

- The queue stores student names who request a book.
- When a student requests a book, we use `enqueue()` to add them to the queue.
- When a book becomes available, we use `dequeue()` to give it to the first student in line.
- This ensures fairness because the first requester gets the book first.

